



UNIVERSIDADE FEDERAL DO CEARÁ – CAMPUS CRATEÚS

Curso: Ciência da Computação

Disciplina: Algoritmos em Grafos

Professor: Rafael Martins Barros

Árvore Geradora Mínima: Análise Comparativa entre Prim e Kruskal

Relatório Técnico

Elislândia Aparecida Horlanda da Silva

Leticia Almeida Lima

Isabelli Araújo Pinho

Crateús – CE, 2026

Sumário

1	Introdução	2
2	Fundamentação Teórica	2
2.1	Definição do problema e representação do grafo	2
2.2	Algoritmo de Prim	3
2.3	Algoritmo de Kruskal	3
3	Método Experimental	4
3.1	Cenários de Teste	4
4	Resultados e Discussão	5
4.1	Grafos esparsos	5
4.2	Grafos densos	6
4.3	Grafos geométricos	7
4.4	Discussão geral	8
5	Conclusão	9
6	Uso de Ferramentas de IA	9

1 Introdução

O problema da Árvore Geradora Mínima (Minimum Spanning Tree - MST) é um dos tópicos mais clássicos e fundamentais em otimização combinatória e teoria dos grafos. Dado um grafo conexo, não direcionado e ponderado $G = (V, E)$, o objetivo é encontrar um subconjunto de arestas que conecte todos os vértices, não contenha ciclos e cuja soma dos pesos seja a menor possível [Cormen et al., 2009]. A resolução desse problema tem ampla aplicação prática no design de redes de computadores, roteamento de telecomunicações, planejamento de malhas elétricas e construção de infraestruturas rodoviárias.

Este relatório documenta a implementação e a análise experimental de dois algoritmos gulosos consagrados para o problema da AGM: o Algoritmo de Prim e o Algoritmo de Kruskal. Sendo o objetivo do trabalho não apenas garantir a corretude das implementações em linguagem C++, mas principalmente investigar o comportamento empírico de ambos os métodos em diferentes condições.

Ao longo das próximas seções, são detalhados os fundamentos teóricos, o delineamento dos experimentos controlados e a discussão crítica dos resultados de desempenho. O foco central reside em observar de forma experimental como as estruturas de dados auxiliares utilizadas por cada algoritmo e a densidade das instâncias (grafos esparsos, densos e geométricos) afetam diretamente o tempo de execução.

2 Fundamentação Teórica

2.1 Definição do problema e representação do grafo

Formalmente, dado um grafo conexo, não direcionado e ponderado $G = (V, E)$, uma Árvore Geradora Mínima é um subconjunto de arestas $T \subseteq E$ tal que (V, T) é uma árvore (conexa e sem ciclos) e a soma dos pesos das arestas em T é a menor possível entre todas as árvores geradoras de G [Kleinberg e Tardos, 2005]. Toda árvore geradora de um grafo com n vértices possui exatamente $n - 1$ arestas. Quando há arestas com pesos repetidos, pode haver mais de uma AGM com o mesmo peso total, mas o peso mínimo é sempre único, esse é um detalhe importante na Seção 4, quando

comparamos os resultados de Prim e Kruskal.

O grafo foi representado por **lista de adjacência**, usando um vetor de vetores (`vector<vector<Aresta>>`), em que cada posição i guarda a lista de arestas que saem do vértice i . Cada aresta armazena o vértice de destino e o peso, sendo este último do tipo `double` para suportar tanto pesos inteiros (grafos esparsos e densos) quanto distâncias euclidianas (grafos geométricos). Como o grafo é não direcionado, cada aresta é inserida duas vezes, uma em cada sentido.

2.2 Algoritmo de Prim

O algoritmo de Prim [Prim, 1957] constrói a AGM de forma gulosa, crescendo uma única árvore a partir de um vértice inicial. A cada passo, escolhe-se a aresta de menor peso que conecta um vértice já incluído na árvore a um vértice ainda fora dela. Para isso, mantém-se um vetor *chave*, que guarda o menor peso conhecido para conectar cada vértice à árvore, e um vetor *pai*, usado para reconstruir as arestas da AGM ao final.

Na implementação, a fila de prioridade foi feita com um `std::set<pair<double,int>`, que mantém os pares (chave, vértice) ordenados automaticamente. Buscar o vértice de menor chave, remover um vértice e atualizar a chave de outro (operação de *decrease-key*, feita como uma remoção seguida de uma nova inserção) custam $O(\log V)$ cada.

2.3 Algoritmo de Kruskal

O algoritmo de Kruskal [Kruskal, 1956] segue uma estratégia diferente: em vez de crescer uma única árvore, ele considera as arestas do grafo em ordem crescente de peso e vai unindo componentes. Uma aresta é adicionada à AGM se, e somente se, seus dois extremos ainda não pertencem ao mesmo componente (ou seja, se adicioná-la não formar um ciclo). Esse controle de componentes é feito com uma estrutura de **conjuntos disjuntos** (*Union-Find*), implementada com as otimizações de *union by rank* e *compressão de caminho* [Cormen et al., 2009], que tornam cada operação de busca ou união praticamente $O(1)$ amortizado [Sedgewick e Wayne, 2011].

O custo dominante do Kruskal é a ordenação das arestas, $O(E \log E)$, já que as operações de Union-Find somadas custam aproximadamente $O(E \alpha(V))$, bem mais barato que a ordenação.

Essa diferença de complexidade explica por que a densidade do grafo deve afetar o desempenho relativo dos dois algoritmos. O Kruskal depende essencialmente de E : em grafos esparsos ($E \approx V$) a ordenação é barata, mas em grafos densos ($E \approx V^2$) o custo $O(E \log E)$ cresce rapidamente. Já o Prim depende da eficiência da fila de prioridade, e seu custo cresce de forma mais suave com E . Espera-se, portanto, que o Kruskal leve vantagem em grafos esparsos e o Prim em grafos densos, comportamento que os experimentos da Seção 4 buscam confirmar.

3 Método Experimental

Para avaliar empiricamente a teoria, ambos os algoritmos foram implementados na linguagem C++. O código foi arquitetado de maneira modular (`grafo.cpp`, `prim.cpp`, `kruskal.cpp`) para isolar responsabilidades. O compilador g++ foi utilizado com a flag de otimização `-O2`.

Os testes foram realizados numa máquina equipada com um processador Intel Core i5 (10^a geração) e o sistema operacional Windows 11. Para cada combinação de tipo de grafo e tamanho V , a instância foi gerada uma única vez e reutilizada nas 10 execuções de cada algoritmo, garantindo o isolamento do tempo de execução em relação ao tempo de construção do grafo. Todas as medições de tempo foram aferidas utilizando a biblioteca `std::chrono::high_resolution_clock`. Além disso, a semente do gerador pseudoaleatório foi fixada (`srand(42)`) para assegurar a reprodutibilidade exata dos grafos gerados. Cabe ressaltar que os tempos absolutos registrados podem sofrer variações dependendo do hardware e da carga do sistema no momento da medição.

3.1 Cenários de Teste

Foram implementados três geradores de grafo, cobrindo diferentes densidades e padrões de peso:

- **Grafos Esparsos:** $|E| \approx |V|$. Gerados criando inicialmente uma corrente entre todos os vértices para garantir conexidade, e adicionando algumas poucas arestas aleatórias. Foram usados grandes volumes de vértices: $V \in \{100, 500, 1000, 5000, 10000\}$.
- **Grafos Densos:** $|E| \approx |V|^2$. Grafos possuindo aproximadamente 80% de todas as arestas possíveis entre os vértices, com pesos aleatórios. Devido ao grande consumo de memória e tempo, as instâncias foram menores: $V \in \{100, 200, 400, 600, 800\}$.

- **Grafos Geométricos:** Vértices distribuídos aleatoriamente no plano 2D bidimensional $[0, 1] \times [0, 1]$, com os pesos das arestas definidos pela distância Euclidiana entre eles. Tamanhos: $V \in \{100, 300, 500, 800, 1000\}$.

O script em Python `gerar_graficos.py` foi posteriormente executado sobre o arquivo gerado `resultados_agm.csv` para a plotagem automática dos gráficos presentes na próxima seção.

4 Resultados e Discussão

As Tabelas 1, 2 e 3 reúnem, para cada instância, o tempo de construção do grafo, o tempo médio de execução de Prim e Kruskal (com desvio padrão de 10 execuções) e o peso final da AGM. Em todos os 15 casos testados, os dois algoritmos chegaram exatamente ao mesmo peso, o que era esperado, mesmo seguindo estratégias diferentes, ambos garantem encontrar uma AGM ótima, e é uma evidência forte de que as duas implementações estão corretas.

4.1 Grafos esparsos

V	Constr. (ms)	Prim (ms)	Kruskal (ms)	Peso
100	0,0475	$0,036 \pm 0,007$	$0,008 \pm 0,003$	4740
500	0,1189	$0,215 \pm 0,020$	$0,061 \pm 0,042$	22570
1 000	0,2035	$0,449 \pm 0,033$	$0,128 \pm 0,014$	45152
5 000	1,3452	$3,036 \pm 0,510$	$0,741 \pm 0,174$	230996
10 000	4,3516	$6,185 \pm 1,981$	$1,246 \pm 0,085$	467158

Tabela 1: Grafos esparsos: tempo de construção, tempo de execução e peso da AGM.

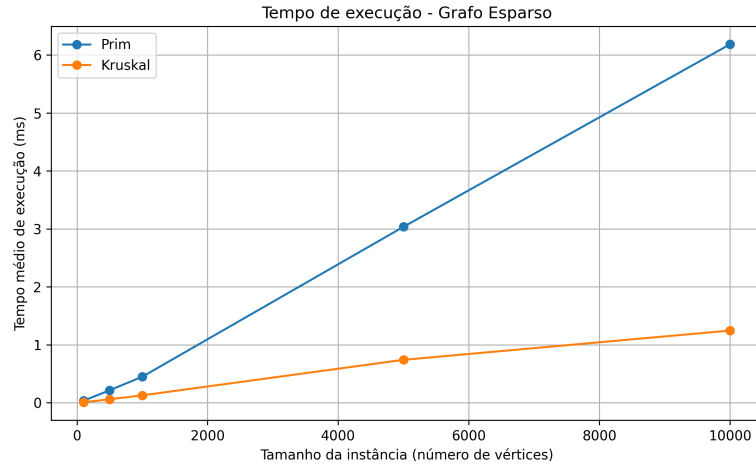


Figura 1: Tempo de execução em função do número de vértices – grafos esparsos.

O tempo de construção cresce quase linearmente com V , como esperado para $|E| \approx V$. Na execução, o Kruskal foi sempre mais rápido que o Prim, com a diferença aumentando junto com V : na maior instância ($V = 10\,000$), o Kruskal levou 1,25 ms contra 6,18 ms do Prim, quase 5 vezes mais rápido. Com poucas arestas, ordenar a lista é barato e o Union-Find resolve o resto quase de graça, enquanto o Prim ainda precisa de V extrações de mínimo em uma estrutura balanceada.

4.2 Grafos densos

V	Constr. (ms)	Prim (ms)	Kruskal (ms)	Peso
100	0,4606	0,183 ± 0,011	0,292 ± 0,031	200
200	1,3689	0,487 ± 0,036	1,109 ± 0,231	251
400	5,8486	1,400 ± 0,305	4,266 ± 0,907	419
600	9,5366	1,943 ± 0,358	9,108 ± 0,641	605
800	18,524	2,825 ± 0,588	20,325 ± 2,021	799

Tabela 2: Grafos densos: tempo de construção, tempo de execução e peso da AGM.

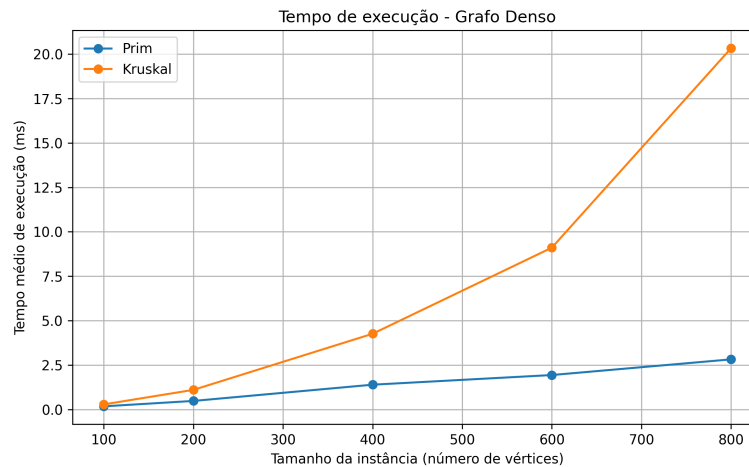


Figura 2: Tempo de execução em função do número de vértices – grafos densos.

O tempo de construção cresce de forma bem mais acentuada que no caso esparsos, por causa do laço que testa todos os $O(V^2)$ pares de vértices. Na execução, o cenário se inverte: o Prim passa a ser sempre mais rápido, e a diferença também cresce com V . Na maior instância ($V = 800$, com cerca de 255 mil arestas), o Kruskal levou 20,33 ms contra apenas 2,82 ms do Prim, mais de 7 vezes mais lento, confirmando que o custo $O(E \log E)$ da ordenação domina quando $E \approx V^2$.

4.3 Grafos geométricos

V	Constr. (ms)	Prim (ms)	Kruskal (ms)	Peso
100	0,7647	0,138 ± 0,011	0,042 ± 0,031	6,472
300	7,1046	0,329 ± 0,018	0,087 ± 0,022	11,442
500	22,330	0,615 ± 0,129	0,144 ± 0,016	14,687
800	52,291	1,081 ± 0,111	0,336 ± 0,022	18,542
1 000	82,020	1,113 ± 0,200	0,335 ± 0,016	20,381

Tabela 3: Grafos geométricos: tempo de construção, tempo de execução e peso da AGM.

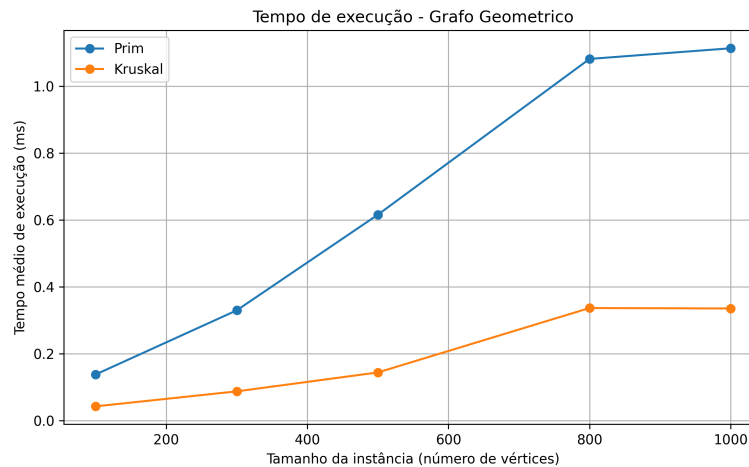


Figura 3: Tempo de execução em função do número de vértices – grafos geométricos.

Apesar de ter poucas arestas por vértice ($K = 4$), o tempo de construção do grafo geométrico é o maior entre os três tipos, a instância com $V = 1\,000$ leva 82,02 ms, mais que o dobro do grafo denso com $V = 800$. Isso acontece porque encontrar os K vizinhos mais próximos de cada vértice exige calcular a distância até todos os outros pontos e ordená-las, um custo de $O(V^2 \log V)$. Já na execução, o comportamento se parece com o do grafo esparsos: como o grau médio é baixo, o Kruskal foi consistentemente mais rápido que o Prim em todos os tamanhos testados, com os tempos do Prim entre $V = 800$ e $V = 1\,000$ praticamente empatados dentro do desvio padrão (1,081 ms e 1,113 ms).

4.4 Discussão geral

Os resultados confirmam as três observações que o enunciado pedia para investigar: o Kruskal depende fortemente da ordenação das arestas (ótimo nos casos esparsos e geométricos, péssimo no denso); o Prim depende da eficiência da fila de prioridade, mantendo-se estável mesmo com muitas arestas; e a densidade do grafo determina qual algoritmo é mais rápido, nenhum dos dois é melhor "no geral". Os desvios padrão observados também foram proporcionais às médias na maioria dos casos, o que é esperado em medições sem isolamento total do sistema operacional.

5 Conclusão

Este trabalho implementou e comparou experimentalmente os algoritmos de Prim e Kruskal para o problema da Árvore Geradora Mínima, utilizando grafos esparsos, densos e geometricamente estruturados, com tamanhos variando de 100 a 10 000 vértices. A validação de corretude mostrou que os dois algoritmos produziram exatamente o mesmo peso total de AGM em todas as 15 instâncias testadas, evidência forte de que as implementações estão corretas.

Os experimentos confirmaram a expectativa teórica: o Kruskal foi até 5 vezes mais rápido que o Prim em grafos esparsos e geométricos, enquanto o Prim foi mais de 7 vezes mais rápido que o Kruskal no grafo denso de maior porte ($V = 800$). Essa inversão de desempenho está diretamente ligada à densidade do grafo, que determina se o custo de ordenar as arestas (Kruskal) ou o custo de manter a fila de prioridade (Prim) domina o tempo total de execução.

6 Uso de Ferramentas de IA

Em conformidade com a Seção 5 do enunciado do trabalho (Portaria N° 39/PRPPG/UFC), a equipe declara o uso de ferramentas de Inteligência Artificial Generativa como apoio pedagógico e de formatação, sem delegação da compreensão crítica ou da geração dos dados experimentais.

- **Ferramentas Utilizadas:** Claude e Google Gemini.
- **Finalidades:**
 1. Estruturação primária de funções auxiliares em C++ para a biblioteca `<chrono>`, garantindo métricas adequadas de nanosegundos/milissegundos.
 2. Auxílio na formatação em $\text{\LaTeX}$ (Overleaf) para alinhar o relatório físico aos padrões de publicação técnica acadêmica.
 3. Consulta sintática para criação correta das chamadas no `gerar_graficos.py` via bibliotecas `matplotlib` e `pandas`.
- **Prompts relevantes utilizados:**
 - “Como configurar adequadamente o `std::chrono` no C++?”

- “Como posso configurar corretamente as bibliotecas de plotagem em python usando arquivos csv?”

Salientamos que em nenhuma etapa houve delegação à IA para fabricar os dados e tempos da `resultados_agm.csv`, bem como interpretar ou gerar análises críticas descritas na Seção 4.

Referências

- [Cormen et al., 2009] Cormen, T. H., Leiserson, C. E., Rivest, R. L., e Stein, C. (2009). *Introduction to Algorithms*. MIT Press, 3. ed.
- [Kleinberg e Tardos, 2005] Kleinberg, J. e Tardos, E. (2005). *Algorithm Design*. Pearson.
- [Kruskal, 1956] Kruskal, J. B. (1956). On the shortest spanning subtree of a graph and the traveling salesman problem. *Proceedings of the American Mathematical Society*, 7(1):48–50.
- [Prim, 1957] Prim, R. C. (1957). Shortest connection networks and some generalizations. *The Bell System Technical Journal*, 36(6):1389–1401.
- [Sedgewick e Wayne, 2011] Sedgewick, R. e Wayne, K. (2011). *Algorithms*. Addison-Wesley, 4. ed.